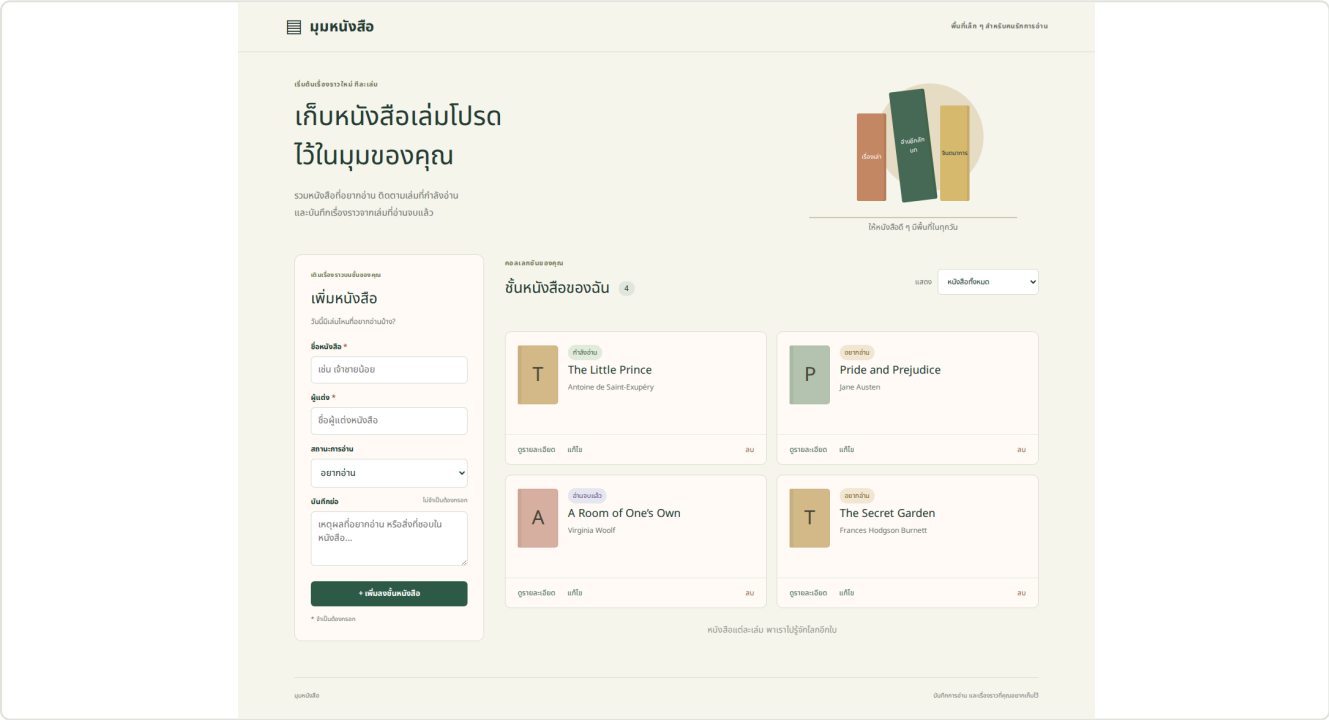
เว็บแอปพลิเคชันจัดการรายการหนังสือ

FULL-STACK CRUD APPLICATION

มุมหนังสือ
Reading List

หัวข้อที่เลือก: รายชื่อหนังสือที่อยากอ่าน กำลังอ่าน และอ่านจบแล้ว

จัดทำเพื่อฝึกออกแบบ REST API และเชื่อมต่อ frontend กับ backend
รองรับ Create, Read, Update และ Delete ผ่าน fetch() โดยไม่รีโหลดหน้าเว็บ



ภาพที่ 1 หน้าเว็บจริง: ฟอร์มเพิ่มหนังสือ รายการหนังสือ และตัวกรองสถานะ

วัตถุประสงค์และขอบเขต

ผู้ใช้บันทึกชื่อหนังสือ ผู้แต่ง สถานะการอ่าน และบันทึกย่อได้ในหน้าเดียว ระบบเก็บข้อมูลในไฟล์ JSON และใช้ Express ให้บริการ API รวมทั้งไฟล์หน้าเว็บ ไม่เรียก API ภายนอก

ผู้จัดทำ / รหัสนักศึกษา / รายวิชา: โปรดเติมก่อนส่งงาน
ไฟล์แก้ไขรายงาน: docs/report.html (เปิดในเบราว์เซอร์ แล้ว Print → Save as PDF)

01 / โครงสร้างระบบและข้อมูล

แบ่งหน้าที่อย่างชัดเจน

ส่วนประกอบ	หน้าที่
public/index.html	ฟอร์มเพิ่มและแก้ไข รายการ ตัวกรอง และหน้าต่างรายละเอียด
public/style.css	สี ตัวอักษร การจัดวาง และหน้าจอ responsive
public/app.js	รับเหตุการณ์จากผู้ใช้ เรียก fetch และอัปเดต DOM
server/app.js	REST routes, validation, อ่านและบันทึก JSON
server/index.js	เริ่มเซิร์ฟเวอร์ที่ port 3000 หรือ PORT ที่กำหนด
data/books.json	ข้อมูลที่คงอยู่แม้ปิดแล้วเปิดเซิร์ฟเวอร์ใหม่

ลำดับการทำงาน

ผู้ใช้กดปุ่ม → JavaScript เรียก fetch → Express ตรวจข้อมูลและดำเนินการกับไฟล์ JSON → ส่ง HTTP status และ JSON กลับ → JavaScript แสดงผลใหม่

Resource: books

ฟิลด์	ชนิด	ข้อกำหนด
id	string	เซิร์ฟเวอร์สร้าง UUID ให้รายการใหม่ แก้ไขไม่ได้
title	string	จำเป็น ความยาว 1–120 หลัง trim
author	string	จำเป็น ความยาว 1–80 หลัง trim
status	string	want-to-read / reading / finished ค่าเริ่มต้น want-to-read
notes	string	ไม่จำเป็น ไม่เกิน 1,000 ตัวอักษร ค่าเริ่มต้น ""

ข้อมูลตัวอย่าง 4 เล่มใช้ ID b1–b4 ส่วนหนังสือที่เพิ่มใหม่จะใช้ UUID
ไฟล์ JSON เหมาะกับงานขนาดเล็กที่รันเซิร์ฟเวอร์หนึ่ง process

การเชื่อมต่อ frontend

ใช้ URL แบบ relative เช่น /api/books ทำให้ frontend และ backend อยู่ต้นทางเดียวกัน ใช้ preventDefault() ในฟอร์ม และรอ API สำเร็จก่อนโหลดรายการใหม่ ข้อความจากผู้ใช้แสดงด้วย textContent

02 / การออกแบบ REST API

Base URL: http://localhost:3000
ข้อมูลที่มี body ใช้ Content-Type: application/json

Method / Endpoint	ผลสำเร็จ	ข้อผิดพลาด
GET /api/books	200 + array ทั้งหมด	500 อ่านข้อมูลไม่ได้
GET /api/books?status=reading	200 + array ที่กรอง ไม่พบคืน []	400 ค่า status ไม่ถูกต้อง
GET /api/books/:id	200 + object	404 ไม่พบ ID
POST /api/books	201 + object ใหม่ และ Location header	400 ข้อมูลไม่ครบหรือผิด
PATCH /api/books/:id	200 + object ที่แก้ไข	404 ไม่พบ ID 400 ข้อมูลผิด
DELETE /api/books/:id	204 ไม่มี body	404 ไม่พบ ID

ตัวอย่างเพิ่มหนังสือ

```
POST /api/books
{"title":"The Little Prince","author":"Antoine de Saint-Exupéry",
 "status":"want-to-read","notes":"Read this weekend"}
```

สำเร็จ: 201 Created และ Location: /api/books/<generated-id>

```
{"id":"generated-uuid","status":"want-to-read",
 "notes":"Read this weekend","title":"The Little Prince",
 "author":"Antoine de Saint-Exupéry"}
```

ตัวอย่างแก้ไขและลบ

```
PATCH /api/books/<id>
{"status":"finished"}

DELETE /api/books/<id> → 204 (empty body)
```

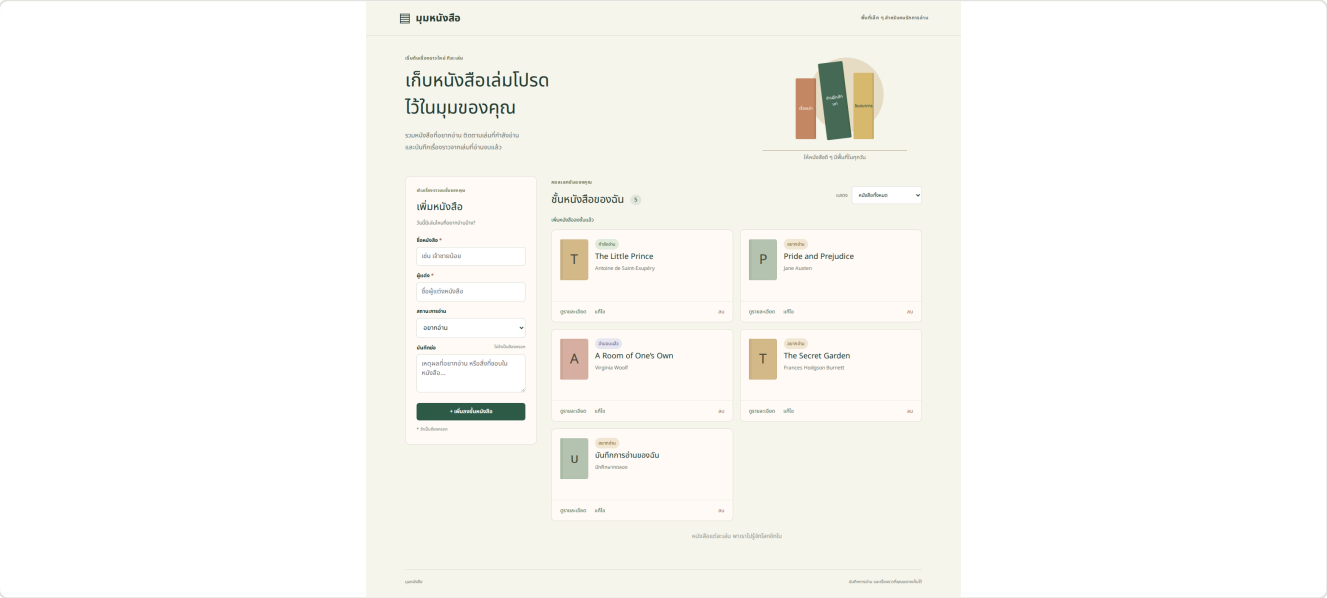
Validation และข้อผิดพลาด

POST บังคับ title และ author; PATCH ต้องส่งอย่างน้อยหนึ่งฟิลด์ที่อนุญาต ตรวจชนิดข้อมูล ความยาว และสถานะ ไม่รับฟิลด์อื่น เช่น id หาก JSON เสีย รูปแบบคืน 400 หาก body เกิน 32 KB คืน 413

```
GET /api/books/missing → 404
{"error":"ไม่พบหนังสือเล่มนี้"}
```

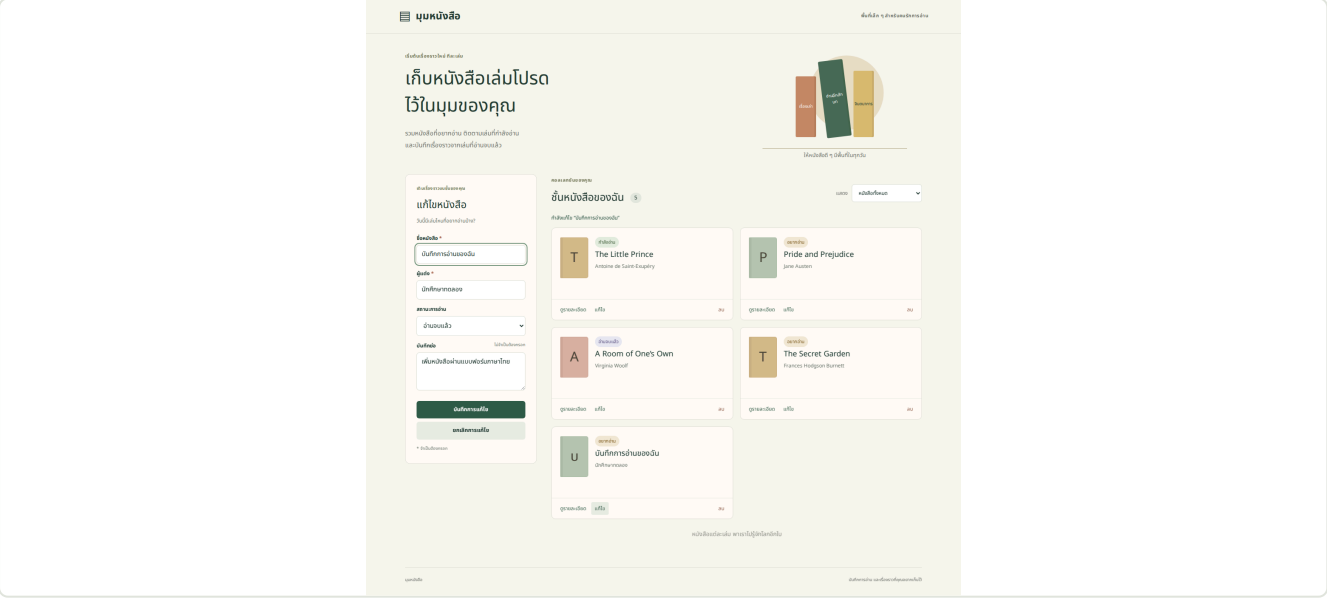
03 / เพิ่มและแก้ไขข้อมูล

กรอกชื่อหนังสือและผู้แต่ง เลือกสถานะ แล้วกด เพิ่มลงชั้นหนังสือ ระบบเรียก POST และโหลดรายการใหม่ด้วย GET หนังสือที่เพิ่มจะแสดงทันทีเมื่อสถานะตรงตัวกรอง



ภาพที่ 2 เพิ่มหนังสือ บันทึกการอ่านของฉัน สำเร็จด้วย POST

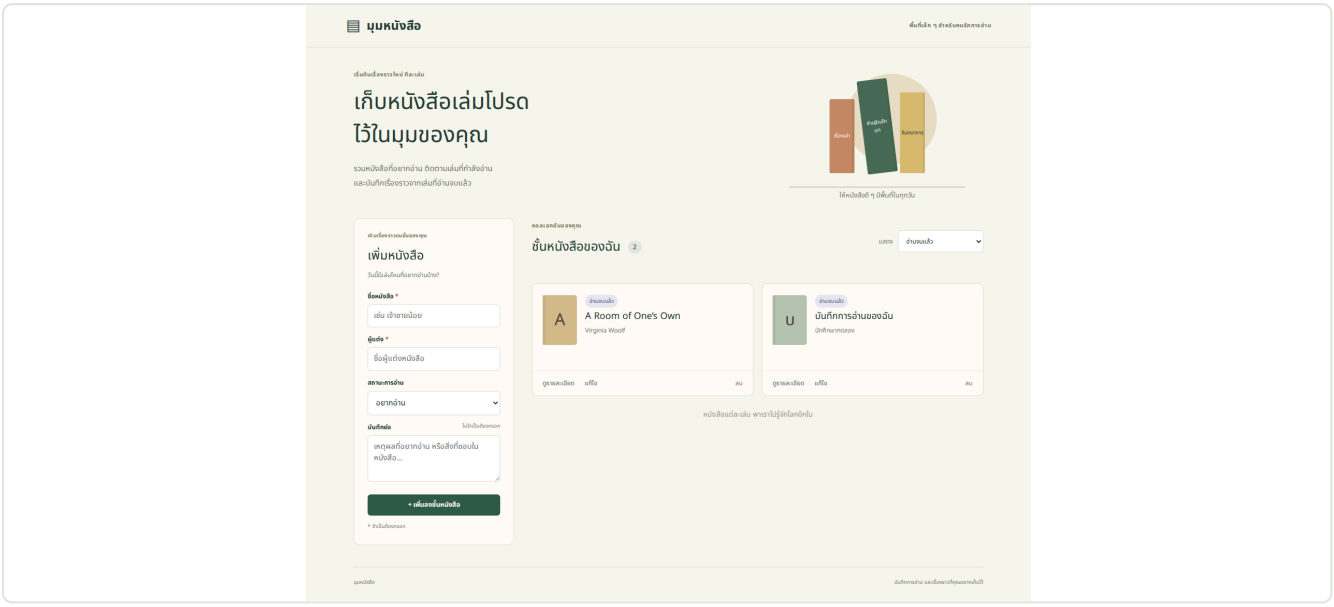
กด แก้ไข เพื่อเรียก GET /api/books/:id และนำข้อมูลใส่ฟอร์ม เมื่อเปลี่ยนสถานะแล้วกด บันทึกการแก้ไข จะส่ง PATCH โดยไม่โหลดหน้าใหม่



ภาพที่ 3 ฟอร์มขณะแก้ไข เลือก อ่านจบแล้ว ก่อนบันทึกด้วย PATCH

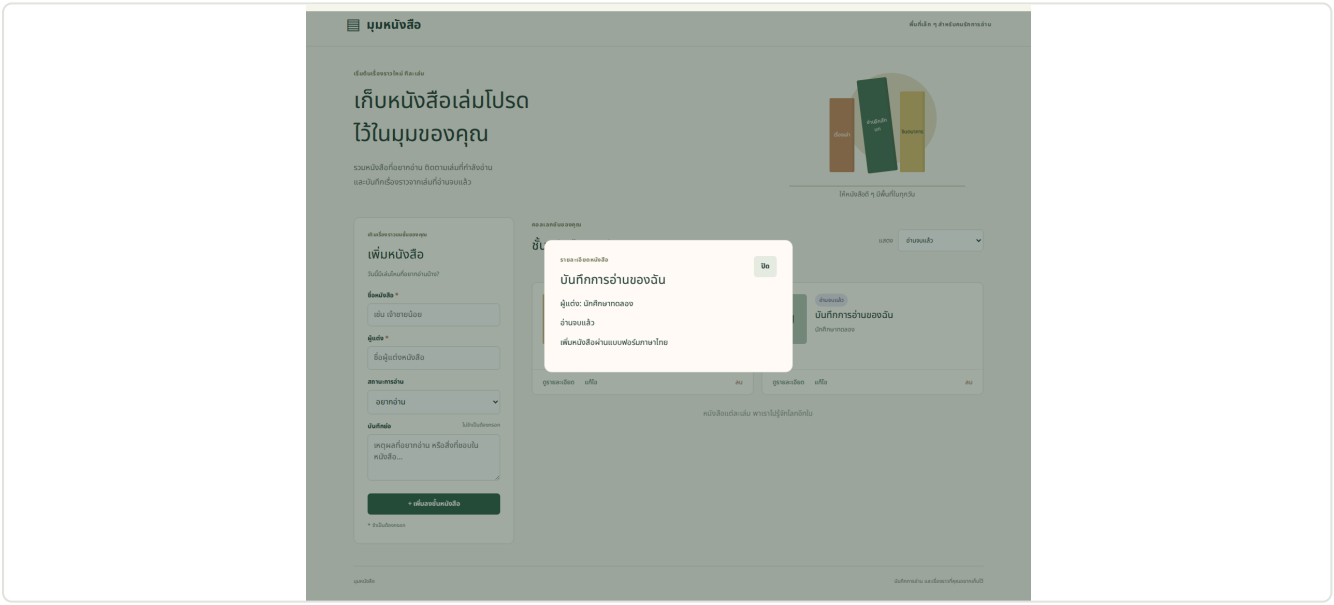
04 / กรองรายการและดูรายละเอียด

เลือก อ่านจบแล้ว จากตัวกรอง ระบบเรียก GET /api/books?status=finished และแสดงเฉพาะหนังสือที่อ่านจบแล้ว



ภาพที่ 4 ผลหลังแก้ไขและกรองสถานะ อ่านจบแล้ว

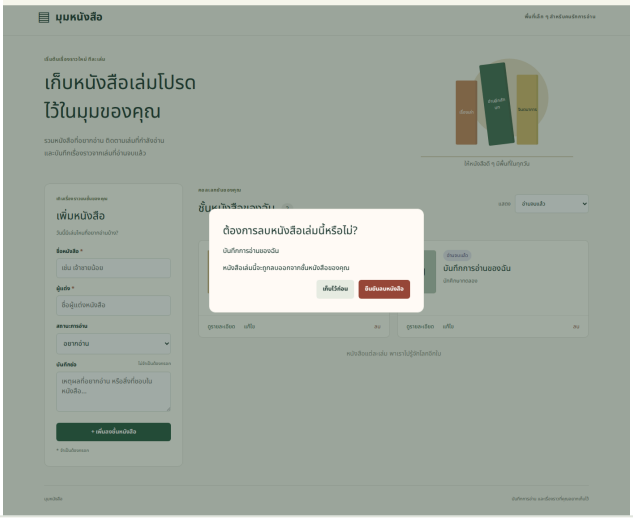
กด ดูรายละเอียด เพื่อดึงข้อมูลหนังสือเล่มเดียวด้วย route parameter และแสดงชื่อ ผู้แต่ง สถานะ และบันทึกย่อใน dialog



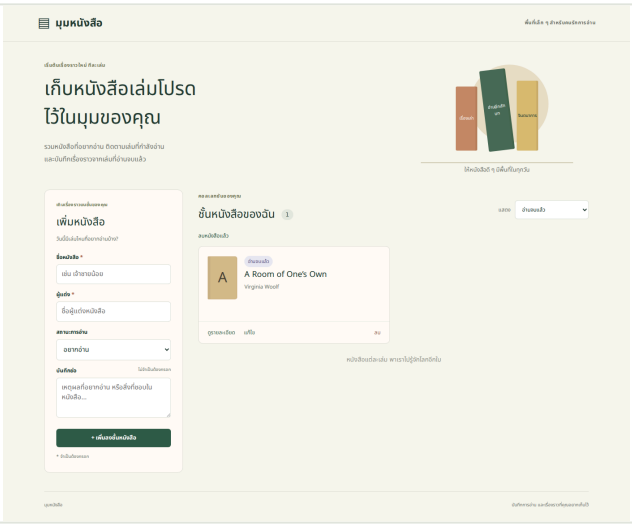
ภาพที่ 5 ข้อมูลที่ดึงจาก GET /api/books/:id

05 / ลบข้อมูลและรองรับจอขนาดเล็ก

กด ลบ แล้วระบบถามยืนยัน เมื่อกด ยืนยันลบหนังสือ จึงเรียก DELETE สำเร็จจะได้ 204 โดยไม่มี response body และรายการถูกอัปเดตทันที



ภาพที่ 6 ถามยืนยันก่อนลบ



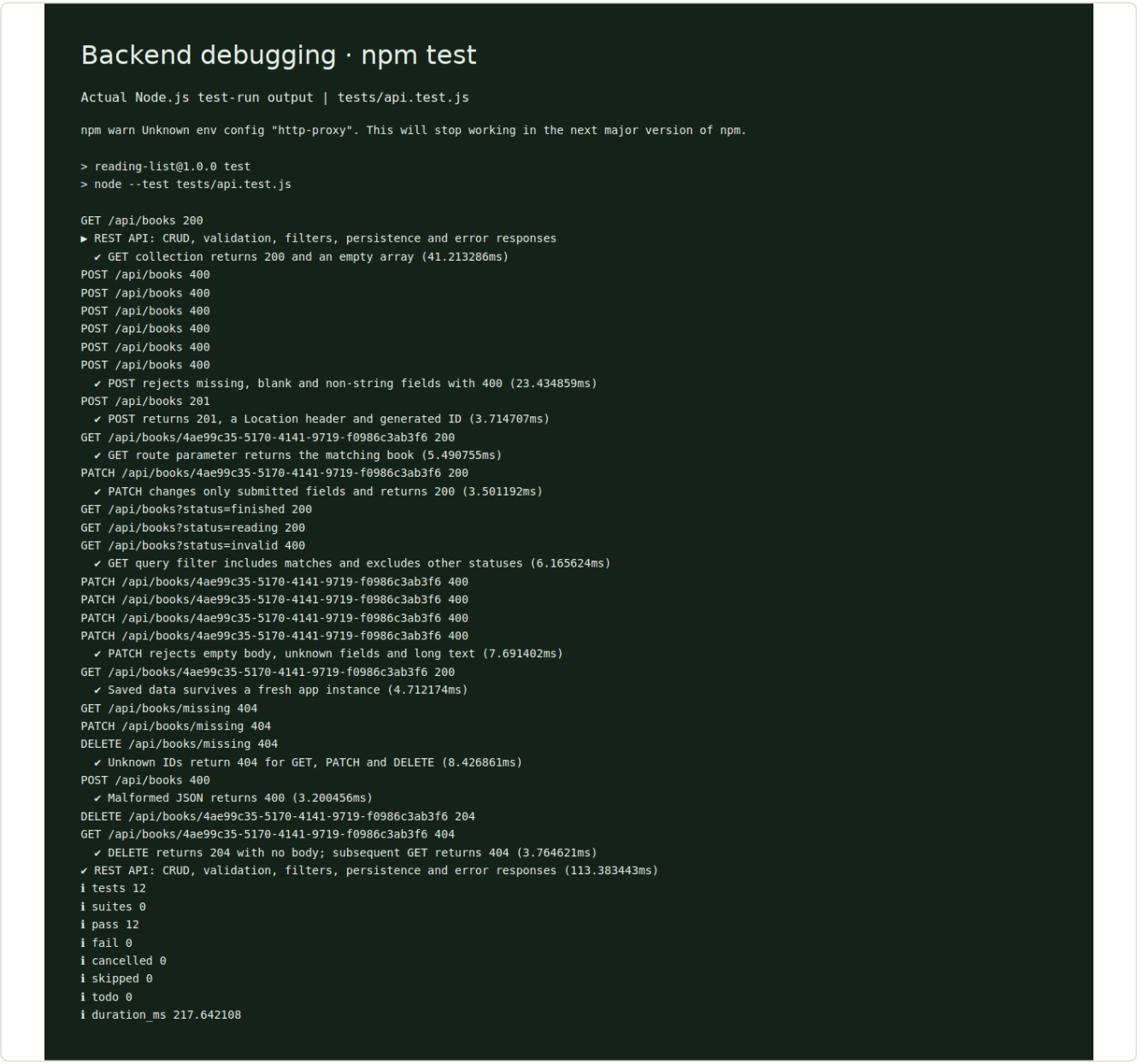
ภาพที่ 7 หนังสือทดสอบถูกลบแล้ว เหลือรายการ อ่านจบแล้ว หนึ่งเล่ม

ตรวจหน้าจอขนาด 390 px แล้วไม่พบการล้นแนวนอน ฟอร์มและรายการเรียงเป็นหนึ่งคอลัมน์ ภาพมือถือฉบับเต็มอยู่ที่ docs/screenshots/08-mobile.png

ผู้ใช้ยกเลิกการลบได้ด้วย เก็บไว้ก่อน หาก API ล้มเหลวระบบแจ้งข้อความผิดพลาด และกรณีโหลดรายการไม่สำเร็จมีปุ่ม ลองอีกครั้ง

06 / หลักฐานดีบักฝั่งเซิร์ฟเวอร์

ทดสอบจริงด้วย npm test บน Node.js v24.19.0 ผลผ่าน 12 tests ตามตัวนับของ Node ได้แก่ 11 กรณีย่อย และ 1 parent test ไม่พบ failure

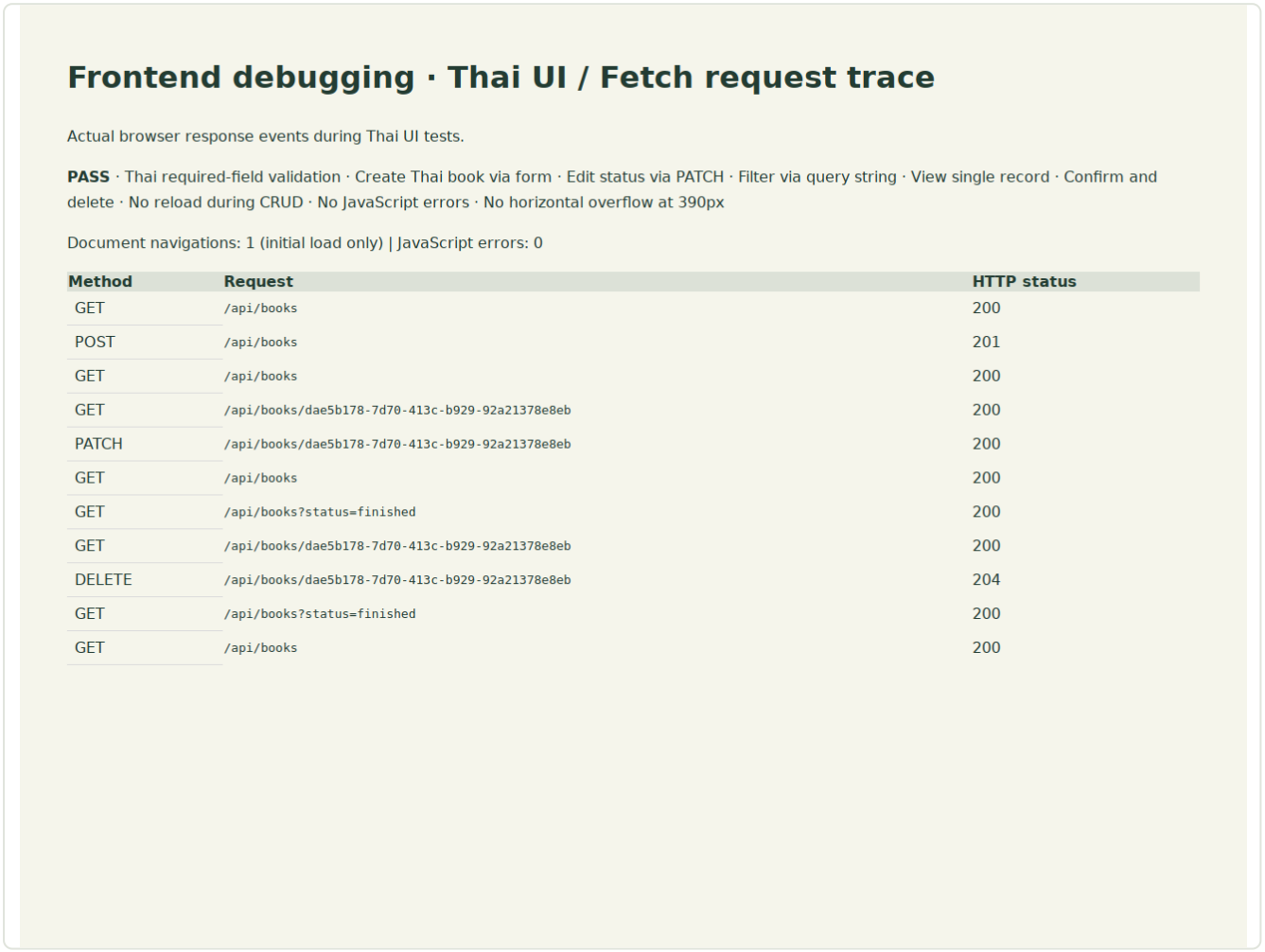


ภาพที่ 8 แสดงข้อความจากการรัน npm test จริง; เก็บข้อความเต็มใน docs/api-test-results.txt

ครอบคลุม CRUD, validation, status filter, ID ที่ไม่พบ, malformed JSON และตรวจว่าข้อมูลอ่านได้จาก app instance ใหม่ การทดสอบใช้ไฟล์ชั่วคราวแยกจาก data/books.json

07 / หลักฐานดีบักฝั่งไคลเอนต์

ใช้เบราว์เซอร์ Chromium ทดสอบการกรอกฟอร์มและกดปุ่มจริง พร้อมบันทึก response events ของคำขอ API ผลผ่านทั้งเพิ่ม แก้ไข กรอง ดูรายละเอียด และลบ



Frontend debugging · Thai UI / Fetch request trace

Actual browser response events during Thai UI tests.

PASS · Thai required-field validation · Create Thai book via form · Edit status via PATCH · Filter via query string · View single record · Confirm and delete · No reload during CRUD · No JavaScript errors · No horizontal overflow at 390px

Document navigations: 1 (initial load only) | JavaScript errors: 0

Method	Request	HTTP status
GET	/api/books	200
POST	/api/books	201
GET	/api/books	200
GET	/api/books/dae5b178-7d70-413c-b929-92a21378e8eb	200
PATCH	/api/books/dae5b178-7d70-413c-b929-92a21378e8eb	200
GET	/api/books	200
GET	/api/books?status=finished	200
GET	/api/books/dae5b178-7d70-413c-b929-92a21378e8eb	200
DELETE	/api/books/dae5b178-7d70-413c-b929-92a21378e8eb	204
GET	/api/books?status=finished	200
GET	/api/books	200

ภาพที่ 9 บันทึกคำขอจริงระหว่างการทดสอบหน้าเว็บ

พบ document navigation 1 ครั้ง คือการเปิดหน้าเว็บครั้งแรก
CRUD ไม่ทำให้ document โหลดใหม่ และไม่พบ JavaScript error ระหว่างทดสอบ

ข้อมูลผลทดสอบอยู่ใน docs/browser-test-results.json และแนบภาพดีบักทั้งสองภาพใน README.md แล้ว

ข้อจำกัด: ภาพดีบักเป็นการแสดงผล test runner และ request trace ที่บันทึกจริง ไม่ใช่ภาพ breakpoint ใน VS Code หรือ Chrome DevTools โดยตรง หากอาจารย์กำหนดภาพชนิดเฉพาะ ต้องจับภาพตามนั้นเพิ่มเติม

08 / ติดตั้ง ทดสอบ และส่งงาน

วิธีเริ่มใช้งาน

ติดตั้ง Node.js 22 ขึ้นไป แตก ZIP แล้วเปิด Terminal ในโฟลเดอร์ที่มี package.json จากนั้นรันทีละคำสั่ง

```
npm install
npm run dev
```

เปิด http://localhost:3000 ในเบราว์เซอร์ ไม่เปิดไฟล์ HTML โดยตรง
กด Ctrl+C เพื่อหยุดเซิร์ฟเวอร์ และใช้ npm test เพื่อตรวจ API

สิ่งที่ส่งใน repository เดียว

รายการ	ไฟล์ / ตำแหน่ง
Backend และ frontend	server/ และ public/
คำแนะนำและ endpoint	README.md
รายงาน PDF และไฟล์แก้ไข	docs/Reading_List_Report.pdf docs/report.html
ภาพใช้งานและหลักฐานดีบัก	docs/screenshots/
Dependency และคำสั่งรัน	package.json และ package-lock.json
แบบทดสอบและผลรัน	tests/ และ docs/*test-results*

อัปโหลด GitHub

สร้าง repository ว่างในบัญชีตนเอง แล้วรันคำสั่งโดยเปลี่ยนชื่อบัญชีและ repository ให้ถูกต้อง

```
git init
git add .
git commit -m "Add Reading List application"
git branch -M main
git remote add origin https://github.com/USERNAME/REPOSITORY.git
git push -u origin main
```

ตรวจว่า README แสดงภาพได้ และมีรายงาน PDF อยู่ใน docs/ ไม่อัปโหลด node_modules ไม่ต้อง host เว็บไซต์ และไม่ใช้ GitHub Pages รัน Node.js

สถานะชุดส่งมอบ: สร้างโค้ดและรายงาน พร้อมผลทดสอบแล้ว
ยังไม่ได้อัปโหลดไปบัญชี GitHub ของนักศึกษา
ก่อนส่ง: เพิ่มข้อมูลผู้จัดทำ และตรวจข้อกำหนดภาพดีบักกับอาจารย์